Formulario de Registro con Flet

---

1. ¿Qué es este proyecto?

Este proyecto es un formulario de registro estudiantil desarrollado con Flet, un framework de Python que permite construir interfaces gráficas de escritorio, web y móvil utilizando únicamente código Python.

El formulario solicita los siguientes datos al usuario:

- Nombre completo
- Número de control
- Correo electrónico
- Carrera (seleccionada con un menú desplegable)
- Semestre (seleccionado con un menú desplegable)
- Género (seleccionado con botones de radio)

Al presionar el botón 'Enviar', el sistema valida todos los campos y, si son correctos, muestra una ventana emergente (AlertDialog) con un resumen de los datos ingresados.

---

2. Requisitos previos

Para ejecutar este proyecto necesitas tener instalado Python 3.8 o superior y la librería Flet. Puedes instalarla con el siguiente comando en tu terminal:

```
pip install flet
```

---

3. Estructura general del código

Todo el código está encapsulado dentro de una función llamada main(page), que es la función principal de cualquier aplicación Flet. Esta función recibe como parámetro un objeto page, que

representa la ventana o página de la aplicación y es el lienzo donde se agrega todo el contenido visual.

Al final del archivo se llama a ft.app() para iniciar la aplicación:

```
ft.app(target=main, view=ft.AppView.WEB_BROWSER)
```

Esto indica que la aplicación se abrirá en el navegador web. Si quisieras abrirla como ventana de escritorio, podrías cambiar el valor de view.

---

4. Configuración inicial de la página

Las primeras líneas dentro de main() configuran el aspecto general de la ventana:

```
page.title = "Formulario de Registro"
page.padding = 30
page.bgcolor = "#FDFBE3"
page.scroll = "auto"
```

Cada propiedad tiene un propósito específico:

- page.title: Define el título que aparece en la pestaña del navegador o en la barra superior de la ventana.
- page.padding: Agrega un espacio de 30 píxeles alrededor de todo el contenido, para que no quede pegado a los bordes.
- page.bgcolor: Establece el color de fondo de la página. El valor #FDFBE3 es un tono crema muy suave.
- page.scroll = "auto": Permite desplazarse verticalmente en la página si el contenido es mayor que la pantalla.

---

5. Controles de entrada de datos

5.1 TextField – Campos de texto

Los TextField son los campos de texto donde el usuario escribe información libremente. Se definen tres campos de este tipo:

Campo: Nombre

```
txt_nombre = ft.TextField(
    label="Nombre",
    border_color="#4D2A32",
    hint_text="Ingresa tu nombre completo",
    keyboard_type=ft.KeyboardType.NAME,
    autofocus=True,
)
```

- label: Es el texto flotante que aparece sobre el campo cuando está activo.
- border_color: Color del borde del campo. #4D2A32 es un tono vino/marrón oscuro.
- hint_text: Es el texto gris de sugerencia que aparece dentro del campo cuando está vacío.

- keyboard_type=ft.KeyboardType.NAME: En dispositivos móviles, activa el teclado optimizado para nombres.
- autofocus=True: Hace que este campo reciba el cursor automáticamente al abrir la aplicación.

Campo: Número de control

```
txt_num_control = ft.TextField(
    label="Número de control",
    keyboard_type=ft.KeyboardType.NUMBER,
)
```

Este campo usa KeyboardType.NUMBER, lo que en dispositivos móviles despliega el teclado numérico. En escritorio el comportamiento es el mismo que un campo normal.

Campo: Email

```
txt_email = ft.TextField(
    label="Email",
    hint_text="correo@ejemplo.com",
    keyboard_type=ft.KeyboardType.EMAIL,
)
```

Al usar KeyboardType.EMAIL en móviles, el teclado incluye el símbolo '@' de forma accesible. La validación real del formato se realiza en la función enviar_datos().

NOTA: El tipo de teclado (keyboard_type) solo afecta dispositivos móviles. En computadoras, el campo acepta cualquier tipo de texto sin importar el keyboard_type.

5.2 Dropdown – Menús desplegables

Los Dropdown presentan al usuario una lista de opciones predefinidas, evitando errores de escritura y estandarizando los datos ingresados.

Dropdown: Carrera

```
dd_carrera = ft.Dropdown(
    label="Carrera",
    expand=True,
    border_color="#4D2A32",
    hint_text="Selecciona tu carrera",
    options=[
        ft.dropdown.Option("Ingeniería en Sistemas Computacionales"),
        ft.dropdown.Option("Ingeniería Industrial"),
        ft.dropdown.Option("Ingeniería Civil"),
        ft.dropdown.Option("Ingeniería Mecánica"),
    ]
)
```

- expand=True: El Dropdown se expande para ocupar todo el espacio disponible en su contenedor padre (en este caso, la mitad del Row donde se coloca junto al de Semestre).
- options: Es la lista de opciones disponibles. Cada ft.dropdown.Option() representa un elemento seleccionable del menú.

Dropdown: Semestre

Funciona de manera idéntica al Dropdown de Carrera, pero sus opciones son los números del 1 al 9, representando cada semestre académico posible. También usa expand=True para ocupar la otra mitad del Row.

5.3 RadioGroup – Selección de género

El RadioGroup permite al usuario elegir exactamente una opción de un conjunto. En este caso se usa para seleccionar el género:

```
rg_genero = ft.RadioGroup(
    content=ft.Row([
        ft.Radio(value="masculino", label="Masculino"),
        ft.Radio(value="femenino", label="Femenino"),
    ]),
)
```

- RadioGroup: Es el contenedor que gestiona la selección. Garantiza que solo se pueda elegir una opción a la vez.
- content: Define el widget visual que contiene los botones de radio. Aquí se usa un Row para mostrarlos horizontalmente.
- ft.Radio(value=..., label=...): Cada Radio es un botón individual. value es el valor interno que se guarda al seleccionarlo, y label es el texto que ve el usuario.

Para leer qué opción fue seleccionada, se accede a rg_genero.value, que devolverá la cadena 'masculino' o 'femenino'.

---

6. Validaciones del formulario

La función enviar_datos(e) se ejecuta cuando el usuario hace clic en el botón 'Enviar'. Esta función implementa un sistema de validación secuencial: revisa cada campo en orden y, si encuentra un error, muestra el mensaje correspondiente y detiene la ejecución con return.

6.1 Limpiar mensajes anteriores

```
mensaje.value = ""
mensaje.color = "red"
```

Antes de validar, se limpia cualquier mensaje de error previo para no confundir al usuario con información antigua.

6.2 Validación: campo vacío

```
if not txt_nombre.value or txt_nombre.value.strip() == "":
    mensaje.value = "❌ El nombre es obligatorio"
    page.update()
    return
```

Esta validación tiene dos partes unidas con or:

- not txt_nombre.value: Verifica si el campo está completamente vacío (None o cadena vacía "").
- .strip() == "": Elimina los espacios en blanco al inicio y al final. Esto evita que el usuario ingrese solo espacios y el formulario lo acepte como válido.

La misma lógica se aplica para txt_num_control. El return corta la ejecución del resto de la función, de manera que solo se muestra un error a la vez.

6.3 Validación: formato de email

```
if not txt_email.value or "@" not in txt_email.value:
    mensaje.value = "❌ Ingresa un email válido"
    page.update()
    return
```

Esta validación verifica dos condiciones:

- not txt_email.value: El campo no puede estar vacío.
- "@" not in txt_email.value: El texto ingresado debe contener el símbolo @, que es parte fundamental de cualquier dirección de correo electrónico.

Si el campo está vacío o no tiene @, se muestra el error y se detiene la ejecución. Esta es una validación básica pero efectiva para filtrar entradas claramente incorrectas.

6.4 Validación: Dropdowns

```
if not dd_carrera.value:
    mensaje.value = "❌ Debes seleccionar una carrera"
    page.update()
    return
```

Los Dropdowns tienen un valor inicial de None (ninguna opción seleccionada). La condición not dd_carrera.value es verdadera cuando el valor es None, lo que significa que el usuario no ha seleccionado ninguna opción. Se aplica la misma lógica para dd_semestre.

6.5 Validación: RadioGroup

```
if not rg_genero.value:
    mensaje.value = "❌ Debes seleccionar un género"
    page.update()
    return
```

De igual forma, rg_genero.value es None si el usuario no ha seleccionado ningún Radio. La condición not rg_genero.value detecta esta situación y muestra el error correspondiente.

---

7. Ventana emergente: AlertDialog

Si todos los campos pasan las validaciones, el código procede a mostrar los datos en una ventana emergente. Esto se logra con ft.AlertDialog.

7.1 Definición del diálogo

```
dialogo_datos = ft.AlertDialog(
    modal=True,
    title=ft.Text("📋 Datos Registrados", ...),
    content=ft.Column([], tight=True, spacing=10),
    actions=[ft.ElevatedButton(...)],
```

```
    actions_alignment=ft.MainAxisAlignment.END,
)
```

- modal=True: Bloquea la interacción con el resto de la aplicación mientras el diálogo está abierto. El usuario debe cerrarlo antes de continuar.
- title: Es el encabezado del diálogo, mostrado en la parte superior.
- content: El cuerpo del diálogo. Se inicia vacío (ft.Column([])) y se rellena dinámicamente con los datos cuando se valida el formulario.
- actions: Botones de acción que aparecen en la parte inferior del diálogo. Aquí hay un botón "Cerrar".
- actions_alignment: Alinea los botones de acción al final (derecha) del diálogo.

7.2 Registro en el overlay

```
page.overlay.append(dialogo_datos)
```

Esta línea es fundamental y obligatoria. Para que un AlertDialog funcione en Flet, debe añadirse al overlay de la página. El overlay es una capa especial que se dibuja por encima de todo el contenido normal, permitiendo que el diálogo aparezca flotando sobre la pantalla.

7.3 Relleno dinámico del contenido

Justo antes de mostrar el diálogo, su contenido se reemplaza con los datos actuales del formulario:

```
dialogo_datos.content = ft.Column([
    ft.Container(content=ft.Column([
        ft.Row([ft.Icon(ft.Icons.PERSON, ...), ft.Text("Nombre:", ...)]),
        ft.Text(txt_nombre.value, ...),
    ]), padding=10, bgcolor='#F0F0F0', border_radius=5),
    # ... más contenedores para cada campo ...
])
```

Cada dato se presenta dentro de un ft.Container con fondo gris claro (#F0F0F0) y esquinas redondeadas (border_radius=5). Dentro de cada contenedor hay un Row con un ícono representativo y la etiqueta del campo, seguidos del valor ingresado por el usuario.

7.4 Abrir el diálogo

```
dialogo_datos.open = True
page.update()
```

Cambiar open a True hace que el diálogo sea visible. La llamada a page.update() actualiza la pantalla para reflejar este cambio inmediatamente.

7.5 Función para cerrar el diálogo

```
def cerrar_dialogo(e):
    dialogo_datos.open = False
    page.update()
```

Esta función se asigna al botón 'Cerrar' del diálogo a través del parámetro on_click. Cuando el usuario hace clic en ese botón, la función cambia open a False y actualiza la página para ocultar el diálogo.

---

8. Botón de envío

```
btn_enviar = ft.ElevatedButton(
    content=ft.Text("Enviar", color="white"),
    bgcolor="#8B7D7D",
    expand=True,
    on_click=enviar_datos,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=5),
    )
)
```

- content=ft.Text("Enviar", color="white"): Define el texto del botón con color blanco para contrastar con el fondo.
- bgcolor="#8B7D7D": Color de fondo del botón, un tono gris rosado.
- expand=True: El botón ocupa todo el ancho disponible en su contenedor.
- on_click=enviar_datos: Conecta el evento de clic con la función de validación. Cada vez que el usuario presione el botón, se llamará a enviar_datos(e).
- RoundedRectangleBorder(radius=5): Da al botón esquinas ligeramente redondeadas.

---

9. Estructura visual (Layout)

La función page.add() agrega un único contenedor principal a la página, que organiza todos los controles:

```
page.add(
    ft.Container(
        content=ft.Column([
            txt_nombre,
            txt_num_control,
            txt_email,
            ft.Row([dd_carrera, dd_semestre], spacing=10),
            ft.Row([ft.Text('Género:'), rg_genero], ...),
            btn_enviar,
            mensaje,
        ], spacing=15, horizontal_alignment=ft.CrossAxisAlignment.STRETCH),
        width=600,
        padding=30,
        border_radius=10,
        bgcolor='white',
        shadow=ft.BoxShadow(blur_radius=10, color='rgba(0,0,0,0.1)'),
    )
)
```

Los contenedores y filas organizan el formulario así:

- ft.Column: Apila todos los controles verticalmente uno tras otro. spacing=15 pone 15 píxeles de espacio entre cada control. horizontal_alignment=STRETCH hace que todos los controles se estiren al ancho completo.

- ft.Row([dd_carrera, dd_semestre]): Pone los dos Dropdowns en la misma fila, separados por 10 píxeles.
- ft.Row([ft.Text('Género:'), rg_genero]): Coloca la etiqueta 'Género:' y los radio buttons en la misma fila.
- ft.Container (externo): Envuelve toda la columna en un rectángulo blanco de 600px de ancho, con padding interno de 30px, bordes redondeados (radius=10) y una sombra suave que le da apariencia de tarjeta elevada.

---

10. Flujo completo de la aplicación

El flujo de uso de la aplicación sigue estos pasos en orden:

- El usuario abre la aplicación y ve el formulario con los campos vacíos.
- El cursor se posiciona automáticamente en el campo 'Nombre' gracias a autofocus=True.
- El usuario completa todos los campos: escribe en los TextField, selecciona opciones en los Dropdown y elige un género con los Radio.
- El usuario hace clic en el botón 'Enviar'.
- Se ejecuta enviar_datos(e). Si hay algún campo inválido, aparece un mensaje de error en rojo y la ejecución se detiene.
- Si todos los campos son válidos, el mensaje cambia a verde ('✅ Datos validados correctamente'), el contenido del AlertDialog se actualiza con los datos del formulario y el diálogo se abre.
- El usuario revisa sus datos en la ventana emergente y hace clic en 'Cerrar' para ocultarla.

---

11. Código completo del proyecto

A continuación se presenta el código completo del proyecto para referencia:

```
import flet as ft

def main(page: ft.Page):
    # Configuración de la página
    page.title = "Formulario de Registro"
    page.padding = 30
    page.bgcolor = "#FDFBE3"
    page.scroll = "auto"

    # --- CONTROLES ---
    txt_nombre = ft.TextField(
        label="Nombre",
        border_color="#4D2A32",
        hint_text="Ingresa tu nombre completo",
        keyboard_type=ft.KeyboardType.NAME,
        autofocus=True,
```

```
)

txt_num_control = ft.TextField(
    label="Número de control",
    border_color="#4D2A32",
    hint_text="Ingresa tu número de control",
    keyboard_type=ft.KeyboardType.NUMBER,
)

txt_email = ft.TextField(
    label="Email",
    border_color="#4D2A32",
    hint_text="correo@ejemplo.com",
    keyboard_type=ft.KeyboardType.EMAIL,
)

dd_carrera = ft.Dropdown(
    label="Carrera",
    expand=True,
    border_color="#4D2A32",
    hint_text="Selecciona tu carrera",
    options=[
        ft.dropdown.Option("Ingeniería en Sistemas Computacionales"),
        ft.dropdown.Option("Ingeniería Industrial"),
        ft.dropdown.Option("Ingeniería Civil"),
        ft.dropdown.Option("Ingeniería Mecánica"),
    ]
)

dd_semestre = ft.Dropdown(
    label="Semestre",
    expand=True,
    border_color="#4D2A32",
    hint_text="Selecciona",
    options=[ft.dropdown.Option(str(i)) for i in range(1, 10)]
)

rg_genero = ft.RadioGroup(
    content=ft.Row([
        ft.Radio(value="masculino", label="Masculino"),
        ft.Radio(value="femenino", label="Femenino"),
    ]),
)

mensaje = ft.Text("", color="red", size=14)

def cerrar_dialogo(e):
    dialogo_datos.open = False
```

```
        page.update()

    dialogo_datos = ft.AlertDialog(
        modal=True,
        title=ft.Text("📋 Datos Registrados", size=22,
weight=ft.FontWeight.BOLD, color="#075E54"),
        content=ft.Column([], tight=True, spacing=10),
        actions=[
            ft.ElevatedButton(
                content=ft.Text("Cerrar", color="white"),
                bgcolor="#075E54",
                on_click=cerrar_dialogo,
            )
        ],
        actions_alignment=ft.MainAxisAlignment.END,
    )

    def enviar_datos(e):
        mensaje.value = ""
        mensaje.color = "red"

        if not txt_nombre.value or txt_nombre.value.strip() == '':
            mensaje.value = "❌ El nombre es obligatorio"
            page.update()
            return

        if not txt_num_control.value or txt_num_control.value.strip() == '':
            mensaje.value = "❌ El número de control es obligatorio"
            page.update()
            return

        if not txt_email.value or "@" not in txt_email.value:
            mensaje.value = "❌ Ingresa un email válido"
            page.update()
            return

        if not dd_carrera.value:
            mensaje.value = "❌ Debes seleccionar una carrera"
            page.update()
            return

        if not dd_semestre.value:
            mensaje.value = "❌ Debes seleccionar un semestre"
            page.update()
            return

        if not rg_genero.value:
            mensaje.value = "❌ Debes seleccionar un género"
```

```
            page.update()
            return

        mensaje.color = "green"
        mensaje.value = "✅ Datos validados correctamente"

        dialogo_datos.content = ft.Column([
            ft.Container(
                content=ft.Column([
                    ft.Row([
                        ft.Icon(ft.Icons.PERSON, color='#075E54', size=20),
                        ft.Text('Nombre:', weight=ft.FontWeight.BOLD, size=14),
                    ], spacing=5),
                    ft.Text(txt_nombre.value, size=14, color='#333333'),
                ], spacing=5),
                padding=10, bgcolor='#F0F0F0', border_radius=5,
            ),
            # ... (se repite el patrón para cada campo)
        ], spacing=10, tight=True, scroll=ft.ScrollMode.AUTO)

        dialogo_datos.open = True
        page.update()

    btn_enviar = ft.ElevatedButton(
        content=ft.Text("Enviar", color="white"),
        bgcolor="#8B7D7D",
        expand=True,
        on_click=enviar_datos,
        style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=5))
    )

    page.add(
        ft.Container(
            content=ft.Column([
                txt_nombre, txt_num_control, txt_email,
                ft.Row([dd_carrera, dd_semestre], spacing=10),
                ft.Row([
                    ft.Text("Género:", size=16, color="#4D2A32",
weight=ft.FontWeight.W_500),
                    rg_genero,
                ], spacing=20, alignment=ft.MainAxisAlignment.START),
                btn_enviar,
                mensaje,
            ],
            spacing=15,
            horizontal_alignment=ft.CrossAxisAlignment.STRETCH,
            ),
            width=600, padding=30, border_radius=10, bgcolor='white',
```

```
            shadow=ft.BoxShadow(spread_radius=1, blur_radius=10,
color='rgba(0,0,0,0.1)'),
        )
    )

    page.overlay.append(dialogo_datos)
    page.update()

ft.app(target=main, view=ft.AppView.WEB_BROWSER)
```